CPE 314 — Software Design

University of Cebu

Computer Engineering Department

Plate N Platter

Plan smarter. Eat better. Cook easier

Project Design Document

Group Name / No.:	PNP/1
Members:	Luistro, Kimberly - Team Leader
	Librea, Nesley Cate - UI
	Nadela, Venice - QA
	Tapan, Yvonne Mae - UI
	Quisagan, Kirk Ryan - Developer
	Tamidles, Ken - DB Admin
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	V 0.2 / October 30, 2026

Table of Contents

Revision History....3
1. Introduction & Project Overview....4
2. Preliminary ER Sketch & Team Roles....5
   2.1 Key attributes and relationships....6
   2.2 Team Roles....6
3. System Architecture & Object-Oriented Design....7
   3.1 Class Hierarchy....7
   3.2 Single Responsibility of Each Class....7
   3.3 Design Rationale....8
4. Data Model & Database Design....9
   4.1 Finalized ERD....9
   4.2 Schema Description....9
5. API Integration Design....11
6. Concurrency Design....11
7. Data Mining & Visualization Design....12
8. Testing Summary....12
9. Individual Contributions....12
Appendix....13

Revision History

Version	Date	Author	Changes
0.1	October 22, 2026	GROUP 1	Initial draft — Week 8 sections
0.2	October 30, 2026	GROUP 1	Week 9 sections added
0.3			Week 10 sections added
1.0			Final compilation and polish — Week 11

1. Introduction & Project Overview

Plate N Platter is a personal meal-planning desktop application for individual users such as students and busy households, built for Option H (Recipe & Meal Planner) of CPE 314. Users register, log in, and plan breakfast, lunch, dinner, and snacks on a calendar where every meal is color-coded by Healthy Level (Healthy, Moderate, or Indulgent) and labeled Quick, Easy, or Hard to prepare. Recipes can be imported from TheMealDB (including Filipino dishes) or created by the user, can be built from other recipes, and feed an automatic shopping list, a cook-next queue, an undo function, and a daily streak that rewards eating at least one healthy meal a day. The app solves the everyday problem of deciding what to cook, knowing what to buy, and staying consistent with healthier eating, all in one simple tool.

Table 2 : Project Summary

ITEM	DETAILS
Theme	Option H — Recipe & Meal Planner
Target users	Individual users (students, busy households); each user has a private account
Platform	Java 17+ desktop application with a JavaFX graphical interface
Data storage	SQLite database accessed through JDBC using PreparedStatement
Public API	TheMealDB (https://www.themealdb.com/api.php), shared test key “1”
Key features	Register/login, calendar with Healthy Level colors, today’s meals with photos, recipe browsing and creation, nested recipes, cook queue, undo, shopping list, streak and badges, weekly statistics with a bar chart

2. Preliminary ER Sketch & Team Roles

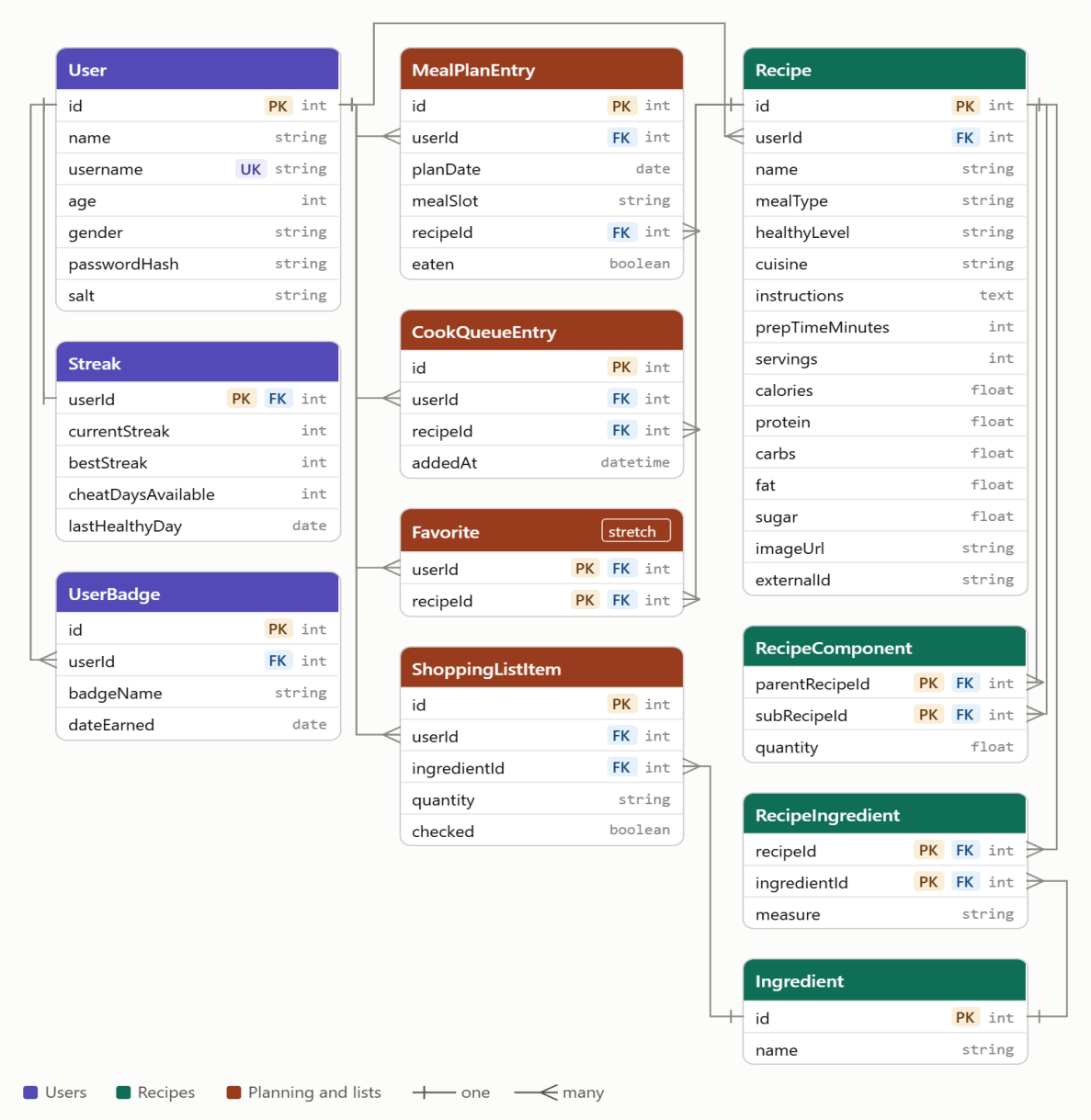


Figure 1: Preliminary ER sketch (entities and key attributes)

2.1 Key attributes and relationships

Table 3: Relationship in the preliminary sketch

From	To	Cardinality	Meaning
Users	Recipes	1 : M	A user creates many recipes
Users	MealPlanEntries	1 : M	A user plans many meals on the calendar
Recipes	MealPlanEntries	1 : M	A recipe can be planned many times
Recipes	Ingredients	M : N	Linked through RecipeIngredient
Recipes	Recipes	M : N	A recipe uses sub-recipes, linked through RecipeComponent
Users	ShoppingListItems	1 : M	Each user has a shopping list
Users	CookQueueEntries	1 : M	Each user has a cook queue
Users	Streaks	1 : 1	One streak record per user
Users	UserBadges	1 : M	A user earns many badges
Users	Recipes (Favorite)	M : N	A user saves favorite recipes (stretch)

2.2 Team Roles

Table 4:Team members and primary roles

Member	Primary Role
Kimberly Luistro	Team Leader
Nesley Cate Librea	User Interface (UI) Developer
Venice Nadela	Quality Assurance (QA) Tester
Yvonne Mae Tapan	User Interface (UI) Developer
Kirk Ryan Quisagan	Software Developer
Ken Tamidles	Database Administrator

3. System Architecture & Object-Oriented Design

3.1 Class Hierarchy

Plate N Platter represents every recipe with one abstract class, Recipe, and three concrete subclasses. Each subclass applies its own prep-time rule.

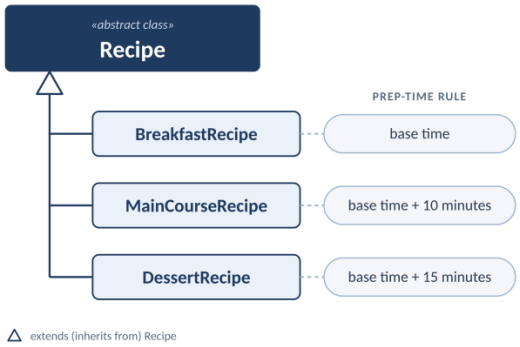


Figure 2: Recipe class hierarchy (abstract class and its three concrete subclasses)

The rest of the application only works with the Recipe type. When the code calls getPrepMinutes() on a Recipe variable, Java runs the version written in the actual subclass. This is polymorphism, and it is used in the running application: the cooking queue, the meal plan, and the calendar all hold Recipe objects without knowing which subclass each one is. A Recipe can also contain other recipes (sub-recipes) to any depth, for example a lasagna that uses a homemade sauce.

3.2 Single Responsibility of Each Class

Table 5: Single Responsibility of each class

Class	Its one responsibility	Why this follows the Single Responsibility Principle
Recipe (abstract)	Holds the data every recipe has (name, cuisine, base prep time, calories, sub-recipes) and declares what every recipe must answer (getPrepMinutes, getMealType).	*It only describes a recipe. It does not total times, save to the database, or draw anything, so it changes only if what a recipe is changes.*
BreakfastRecipe	Knows the breakfast prep-time rule and names its meal type.	*The only reason to edit it is a change to the breakfast rule.*
MainCourseRecipe	Knows the main-course prep-time rule (base time + 10 minutes).	*The only reason to edit it is a change to the main-course rule.*
DessertRecipe	Knows the dessert prep-time rule (base time + 15 minutes).	*The only reason to edit it is a change to the dessert rule.*
RecipeCalculator	Totals prep time and calories through nested sub-recipes, and	*Calculation is kept out of Recipe so the recipe classes stay simple*

	detects circular references.	*data classes.*
CookingQueue	Keeps the order recipes will be cooked (first in, first out).	*It manages one queue and knows nothing about the screen or database.*
UndoStack<T>	Remembers actions so the latest one can be undone (last in, first out).	*It is a generic stack and does not know what kind of action it stores.*
MealPlanEntry	Represents one planned meal (date, meal slot, recipe, eaten or not).	*It only stores one plan item.*
MealPlanBook	Groups planned meals by date using a Map.	*It only answers "what is planned on this day?".*
Database	Opens the SQLite connection and creates the tables from schema.sql.	*Connection setup is in one place, so no other class repeats it.*
UserDao	Runs the SQL for the users table using PreparedStatement.	*It is the only class that talks to that table.*
User	Holds one user's data (with a password hash, never the password).	*It only stores data.*
PasswordHasher	Creates salts and hashes passwords.	*Security code lives in one class and can be tested alone.*
RegisterScreen	Shows the sign-up form and handles its button	*It checks the form and calls UserDao; it does not contain SQL*
CalendarScreen	Shows the monthly calendar and the meals of the clicked day.	*It only displays; the data comes from MealPlanBook.*
App	Starts the database and opens the window.	*It only starts the program and connects the screens.*

3.3 Design Rationale

We used an abstract class instead of one **Recipe** class with a "type" field because every recipe type shares the same data but follows a different prep-time rule. Putting each rule in its own subclass means a new rule changes one small class, and the code that uses recipes never needs if statements about the type. We kept calculations, storage, and screens in separate classes (**RecipeCalculator**, **UserDao**, the **ui** classes) so that each part can be built and tested by a different team member without breaking the others, and so that unit tests can check the logic without opening a window or a database.

4. Data Model & Database Design

4.1 Finalized ERD

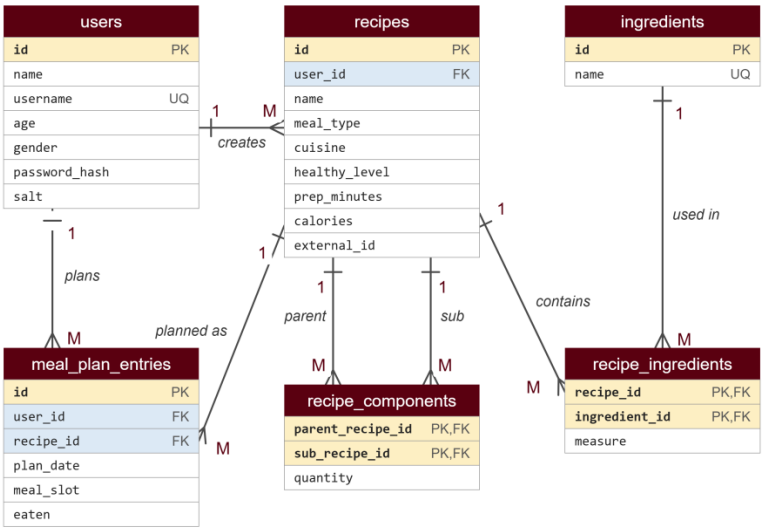


Figure 3: Finalized entity-relationship diagram

Table 6: Relationships and cardinality

Relationship	Cardinality	Implemented by
users and recipes	1:M	**recipes.user_id** (one user can create many recipes; imported recipes may have no user)
users and meal_plan_entries	1:M	**meal_plan_entries.user_id**
recipes and meal_plan_entries	1:M	**meal_plan_entries.recipe_id**
recipes and recipes (sub-recipes)	M:M	junction table **recipe_components** (**parent_recipe_id**, **sub_recipe_id**)
recipes and ingredients	M:M	junction table **recipe_ingredients** (**recipe_id**, **ingredient_id**)

4.2 Schema Description

The schema is stored in **src/main/resources/schema.sql** in the repository and is run by **Database.initialize()** every time the application starts. Each statement uses **CREATE TABLE IF NOT EXISTS**, so running it again is safe.

Table 7: Tables in the database

The CREATE TABLE Statements:

```
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    username TEXT NOT NULL UNIQUE,
    age INTEGER,
    gender TEXT,
    password_hash TEXT NOT NULL,
    salt TEXT NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS recipes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    name TEXT NOT NULL,
    meal_type TEXT NOT NULL,
    cuisine TEXT,
    healthy_level TEXT,
    prep_minutes INTEGER NOT NULL DEFAULT 0,
    calories INTEGER NOT NULL DEFAULT 0,
    external_id TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

```
CREATE TABLE IF NOT EXISTS recipe_components (
    parent_recipe_id INTEGER NOT NULL,
    sub_recipe_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL DEFAULT 1,
    PRIMARY KEY (parent_recipe_id, sub_recipe_id),
    FOREIGN KEY (parent_recipe_id) REFERENCES recipes(id),
    FOREIGN KEY (sub_recipe_id) REFERENCES recipes(id)
);
```

```
CREATE TABLE IF NOT EXISTS ingredients (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE
);
```

```
CREATE TABLE IF NOT EXISTS recipe_ingredients (
    recipe_id INTEGER NOT NULL,
    ingredient_id INTEGER NOT NULL,
    measure TEXT,
    PRIMARY KEY (recipe_id, ingredient_id),
    FOREIGN KEY (recipe_id) REFERENCES recipes(id),
    FOREIGN KEY (ingredient_id) REFERENCES ingredients(id)
);
```

```
CREATE TABLE IF NOT EXISTS meal_plan_entries (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    recipe_id INTEGER NOT NULL,
    plan_date TEXT NOT NULL,
    meal_slot TEXT NOT NULL,
    eaten INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (recipe_id) REFERENCES recipes(id)
);
```

4.3 Design Notes

- **Junction tables.** Sub-recipes and ingredients are many-to-many, so each has its own table. **recipe_components** is what lets the recursive calculator in Section 3.4 follow recipes inside recipes.
- **Passwords.** The users table stores only **password_hash** and **salt**. The real password is never saved.
- **Dates.** SQLite has no date type, so **plan_date** is stored as text in the form YYYY-MM-DD. The **eaten** column stores 0 or 1.
- **Foreign keys.** SQLite ignores foreign keys unless they are switched on, so **Database.getConnection()** runs **PRAGMA foreign_keys = ON** for every connection.
- **Safe SQL.** All JDBC statements use **PreparedStatement** with bound parameters.
- **Planned for Milestone 2.** The streak, badge, favorites, shopping-list, and cook-queue tables from the Week 8 sketch are not part of this schema yet. They will be added to **schema.sql** and to this section when they are built. For now the cooking queue and the undo stack are kept in memory.

5. API Integration Design

Name the public API your app consumes, describe the endpoint(s) called, the shape of the JSON response, and how that data is parsed and used meaningfully in the app.

6. Concurrency Design

Identify the long-running operation(s) that run on a background thread, explain the threading approach used, and describe how the UI stays responsive during that operation.

7. Data Mining & Visualization Design

Describe the aggregation or frequency-analysis step performed on your app's data and the chart(s) used to visualize the result, including why that chart type was chosen.

8. Testing Summary

Summarize your JUnit test suite — how many tests exist, what they cover, and at least one meaningful edge case per tested class. A short table or list of key test cases is helpful here.

9. Individual Contributions

List what each team member specifically built or owned, referencing their git commit history, so the instructor can cross-check contribution during the Individual Q&A.

Appendix

(Add any additional references, links to full API documentation, licensing notes, or supplementary material here.)